Teamarbeit Temp

Was bedeutet „Pages for public facing content“ konkret?

GitHub Pages ist ein **Static-Site-Hosting** direkt aus einem Repository: Du legst HTML/CSS/JS oder Markdown (optional mit Build/Generator) ins Repo, GitHub baut/publisht daraus eine Website. Typisch ist eine URL wie `https://<user>.github.io/<repo>/` oder bei User/Org-Pages `https://<user>.github.io/`.

Public facing content heisst: Inhalte, die **für Aussenstehende** gedacht sind (Kunden, Recruiter, Open-Source-User, andere Teams) – also mehr als nur „Entwickler-notizen“, sondern **sauber präsentierte, navigierbare Doku/Website**.

Markdown Variante -> z.B. mit MkDocs, Docusaurus, Jekyll (GitHub-Standard), Hugo -> man tut dann z.b.

```
mkdocs build
```

ausführen und dann werden die markdown files in html umgewandelt.
Warum: Markdowns beschreiben struktur und Inhalt, nicht Layout -> Es ist einfacher, kürzer, lesbarer und weniger fehleranfällig mit markdown zu arbeiten

Beispiele ->
In Markdown:

```
# Installation
Run `npm install`
```

In HTML:

```
<h1>Installation</h1>
<p>Run <code>npm install</code></p>
```

Wofür ist GitHub Pages gut?

1) Projekt-Website statt nur Repo-Ansicht

- Landing Page: „Was ist das?“, Screenshots, Features, Demo-Links, Roadmap
- „Getting Started“ für neue Nutzer (Install, Setup, Quickstart)
- API-/Architektur-Doku als eigenständige Site, besser lesbar als ein einzelnes README

2) Versionierte Doku „as Code“

- Inhalte leben im Repo → Pull Requests, Reviews, Issues, Versionierung
- Releases/Tags können zu Doku-Versionen werden (je nach Generator/Workflow)

3) Team-/Kurs-Kontext (unser Use Case)

Für unser Modul: Eine Pages-Site ist ideal, um die Inhalte **wie ein Mini-Wiki mit Navigation** aufzubauen, aber **schöner für Konsumenten** (Dozent, Klasse) als ein Wiki-Tab oder lange README.

Was muss man beachten (kritische Punkte)?

A) GitHub Pages ist „static“

- Keine serverseitige Logik (keine DB, keine geheimen Tokens im Backend)
- Alles, was im ausgelieferten HTML/JS steht, ist öffentlich einsehbar

B) Keine sensiblen Inhalte

GitHub selbst warnt sinngemäss: **nicht für sensitive Transaktionen** (Passwörter, Zahlungsdaten etc.). Und: **Pages ist öffentlich erreichbar**, selbst wenn dein Repo (je nach Plan) privat sein kann.

C) Publishing Source / Build-Strategie

Du entscheidest, **woher** Pages veröffentlicht:

- **Branch + Ordner**: z.B. `main` + `/docs` oder `main` + `/` (Root)
- **GitHub Actions Workflow**: wenn du Kontrolle über Build brauchst (MkDocs, Docusaurus, etc.)

D) Sichtbarkeit / Pläne

- Public Repos: Pages ist mit Free verfügbar
- Private Repos: Pages hängt am Plan (Pro/Team/Enterprise etc.).

Vergleich: README vs Wiki vs Pages (wann was?)

README (Repo-Entry, „first contact“)

Stärken

- Sofort sichtbar beim Öffnen des Repos
- Ideal für Quickstart, Setup, wichtigste Links (auch Link auf Pages!)
- Ein File, einfache Pflege

Schwächen

- Wird schnell zu lang
- Navigation/IA (Information Architecture) begrenzt

Typisch

- „Was ist es?“ + „Wie starte ich?“ + „Links zu Doku/Pages/Wiki“

Wiki (internes/halb-internes Wissenslager)

Stärken

- Viele Seiten, schnelle Notizen, kollaborativ
- Gut für: Meeting Notes, Entscheidungslogs, Konzepte, Team-Wissen

Schwächen

- Weniger „Produkt-Website“-Feeling
- Oft schlechter versioniert/gekoppelt an Code-Reviews (je nach Workflow)

Pages (public facing, strukturiert & präsentierbar)

Stärken

- Beste UX für Leser: Navigation, Suchfunktion (mit Tools), klare Struktur
- Perfekt für: Benutzer-Doku, Architektur-Doku, Onboarding, Projektpräsentation
- Kann aus Markdown generiert werden (MkDocs etc.) und automatisiert deployen

Schwächen

- Etwas Setup-Aufwand
- Build/Publishing muss sauber laufen

Pragmatische Regel

- README = Einstieg + Wegweiser
- Pages = „die eigentliche Doku/Website“
- Wiki = internes Wissen/Notizen/Entscheidungen

Sinnvolle Überthemen für eine Pages-Doku

1. **Home / Overview**

- Kurzbeschreibung, Ziel, für wen
- 30-Sekunden-Zusammenfassung, ggf. Screenshot/GIF
- Links: Repo, Wiki, Issues, Releases

2. **Getting Started**

- Voraussetzungen
- Installation/Setup
- Quickstart (minimaler Pfad, der funktioniert)
- Troubleshooting (Top 5 Fehler)

3. **User Guide**

- Kernfunktionen nach Use Cases
- Beispiele (Screenshots, Sequenzen)

4. **Architecture**

- Kontext: Problem, Constraints
- Komponentenübersicht (Diagramm)

- Datenfluss / Schnittstellen (API, Events)
- Deployment/Environment (bei euch z.B. Dev/Prod)

5. **Contribution / Collaboration**

- Branching-Konventionen
- PR-Regeln
- Code Style, Commit Messages
- „How to contribute“

6. **FAQ**

- Wiederkehrende Fragen kompakt beantwortet

7. **Changelog / Releases**

- Was hat sich geändert, Breaking Changes

8. **Security / Privacy (kurz, aber wichtig)**

- Was gehört nicht in Pages
- Umgang mit Secrets
- Responsible Disclosure (falls relevant)

Beispiel: Wie könnte eine Page strukturiert sein?

Minimal (ohne Generator): `docs/` mit Markdown

Struktur im Repo

```
/docs
  index.md
  getting-started.md
  user-guide.md
  architecture.md
  contribution.md
  faq.md
  assets/
    images/
```

`docs/index.md` (Inhaltsskizze)

- Titel + 2–3 Sätze „Was ist das Projekt?“
- „Für wen ist es?“ (Zielgruppe)
- „Was kann es?“ (Bulletpoints)
- Quick Links (Repo, Issues, Wiki)
- „Start in 3 Schritten“
- Optional: Screenshot/Diagramm (aus `/docs/assets/images`)
- Abschnitt „Dokumentations-Navigation“ (Links zu den Seiten)

Möglichkeiten Infos darzustellen

- Tabellen (z.B. Feature-Matrix)
- Code-Blöcke (Install-Kommandos)
- Mermaid-Diagramme *nur*, wenn euer Renderer das unterstützt (sonst als Bild)
- Bilder/GIFs (UI-Demo)
- Callouts (Hinweise/Warnings) – abhängig vom Generator/Theme

Professioneller (mit MkDocs)

- Navigation links
- Suche
- Thema/Branding
- Deploy via Actions (passt zu „MkDocs via Actions“)

Step-by-Step: GitHub Pages in einem Repo erstellen (einfachster Weg)

Variante 1: Ohne Build, direkt aus Branch + `/docs`

1. **Im Repo** einen Ordner `docs/` erstellen.
2. In `docs/` eine Startseite anlegen:
   - `index.md` (oder `index.html`)
3. Auf GitHub: **Repo** → **Settings** → **Pages**
4. Unter **Build and deployment**:
   - **Source**: „Deploy from a branch“
   - **Branch** wählen (meist `main`)
   - **Folder**: `/docs`
5. Speichern → GitHub generiert die Pages-URL und baut die Site.

Wichtig: Wenn du `index.md` nutzt, hängt das Rendering von der Pages-Pipeline/Theme ab; mit `index.html` bist du maximal deterministisch.
https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

Variante 2: Mit GitHub Actions (für MkDocs/Docusaurus/etc.)

1. Doku-Generator konfigurieren (z.B. MkDocs erzeugt eine `site/` Ausgabe).
2. Workflow YAML in `.github/workflows/<name>.yml` anlegen.
3. In **Settings** → **Pages**: Source auf **GitHub Actions** stellen.
4. Push → Workflow baut und deployed automatisch.